

Объектно-ориентированное программирование

Никита Остроухов

Кафедра информатики МФТИ

Библиотека 2D математики

Векторная математика

Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Умножение

$$\begin{Bmatrix} x \\ y \end{Bmatrix} * N = \begin{Bmatrix} x * N \\ y * N \end{Bmatrix}$$

Вычитание

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} - \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 - x2 \\ y1 - y2 \end{Bmatrix}$$

Деление

$$\begin{Bmatrix} x \\ y \end{Bmatrix} / N = \begin{Bmatrix} x / N \\ y / N \end{Bmatrix}$$

Векторная математика

Скалярное умножение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} * \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = x1 * x2 + y1 * y2$$

Длина

$$\text{len} \begin{Bmatrix} x \\ y \end{Bmatrix} = \text{sqrt}(x^2 + y^2)$$

Нормализация

$$\text{normalize} \begin{Bmatrix} x \\ y \end{Bmatrix} = \begin{Bmatrix} x / \text{sqrt}(x^2 + y^2) \\ y / \text{sqrt}(x^2 + y^2) \end{Bmatrix}$$

Реализация библиотеки



Будем работать в 2D пространстве для
упрощения математических вычислений

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

Будем работать в 2D пространстве для упрощения математических вычислений

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

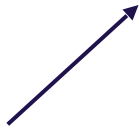
Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
... add ( ... );
```



Аргументы?

Сложение

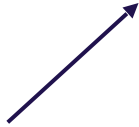
$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
... add ( ... );
```

Аргументы?



Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Участвуют два
вектора



Реализация библиотеки

```
struct Vector2f
{
    float x;
    float y;
};

... add (Vector2f v1, Vector2f v2);
```

Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
... add (Vector2f v1, Vector2f v2);
```



**Возвращаемое
значение?**

Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
... add (Vector2f v1, Vector2f v2);
```

Возвращаемое
значение?



Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Вектор как
результат операции



Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
Vector2f add (Vector2f v1, Vector2f v2);
```

Сложение

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} + \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 + x2 \\ y1 + y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
Vector2f add (Vector2f v1, Vector2f v2);  
Vector2f sub (Vector2f v1, Vector2f v2);
```

Вычитание

$$\begin{Bmatrix} x1 \\ y1 \end{Bmatrix} - \begin{Bmatrix} x2 \\ y2 \end{Bmatrix} = \begin{Bmatrix} x1 - x2 \\ y1 - y2 \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
Vector2f add (Vector2f v1, Vector2f v2);  
Vector2f sub (Vector2f v1, Vector2f v2);  
Vector2f mul (Vector2f v1, float n);  
Vector2f div (Vector2f v1, float n);
```

Умножение

$$\begin{Bmatrix} x \\ y \end{Bmatrix} * N = \begin{Bmatrix} x * N \\ y * N \end{Bmatrix}$$

Деление

$$\begin{Bmatrix} x \\ y \end{Bmatrix} / N = \begin{Bmatrix} x / N \\ y / N \end{Bmatrix}$$

Реализация библиотеки

```
struct Vector2f  
{  
    float x;  
    float y;  
};
```

```
Vector2f add (Vector2f v1, Vector2f v2);  
Vector2f sub (Vector2f v1, Vector2f v2);  
Vector2f mul (Vector2f v1, float n);  
Vector2f div (Vector2f v1, float n);
```

```
float scalar (Vector2f v1, Vector2f v2);  
float length (Vector2f v);
```

```
Vector2f normalize (Vector2f v);
```

Остальные векторные операции

Реализация библиотеки

```
Vector2f add (Vector2f v1, Vector2f v2)
{

}
```

Рассмотрим реализацию функции **add**

Реализация библиотеки

```
Vector2f add (Vector2f v1, Vector2f v2)
{
    Vector2f v = {v1.x + v2.x, v1.y + v2.y};
    return v;
}
```

Рассмотрим реализацию функции **add**

Реализация библиотеки

```
Vector2f add (Vector2f v1, Vector2f v2)
{
    Vector2f v = {v1.x + v2.x, v1.y + v2.y};
    return v;
}
```

Для удобства дальнейшей работы создадим
функцию **Vector2f_ctor**

Реализация библиотеки

```
Vector2f Vector2f_ctor (float x, float y)
{
    Vector2f v = {x, y};
    return v;
}

Vector2f add (Vector2f v1, Vector2f v2)
{
    Vector2f v= {v1.x + v2.x, v1.y + v2.y};
    return v;
}
```

Для удобства дальнейшей работы создадим функцию **Vector2f_ctor**

Реализация библиотеки

```
Vector2f Vector2f_ctor (float x, float y)
{
    Vector2f v = {x, y};
    return v;
}

Vector2f add (Vector2f v1, Vector2f v2)
{
    return Vector2f_ctor(v1.x + v2.x, v1.y + v2.y);
}
```

Функцию **создания** вектора удобно
использовать

Задание

Реализовать остальные функции для вектора

Реализация библиотеки

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\text{normalize}(v1 + v2 + v3) * \text{len}(v2 + v3)$$

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

$$\mathbf{v1} = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad \mathbf{v2} = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad \mathbf{v3} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

normalize(v1 + v2 + v3) * len(v2 + v3)

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
add(v1, v2)
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

normalize(v1 + v2 + v3) * len(v2 + v3)



Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
add(add(v1, v2), v3)
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

normalize(v1 + v2 + v3) * len(v2 + v3)



Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(add(add(v1, v2), v3))
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\underline{\text{normalize}(v1 + v2 + v3)} * \text{len}(v2 + v3)$$



Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(add(add(v1, v2), v3))  
length(add(v2, v3))
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\underline{\text{normalize}(v1 + v2 + v3)} * \underline{\text{len}(v2 + v3)}$$


Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
mul(normalize(add(add(v1, v2), v3)),  
      length(add(v2, v3)))
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

$$\underline{\text{normalize}(v1 + v2 + v3) * \text{len}(v2 + v3)}$$


Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
mul(normalize(add(add(v1, v2), v3)),  
      length(add(v2, v3)))
```

Работать с такими выражениями невозможно

Как упростить?

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;
};

Vector2f add (Vector2f v1, Vector2f v2)
{
    return Vector2f_ctor(v1.x + v2.x, v1.y + v2.y);
}
```

Рассмотрим реализацию еще раз

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;
};

Vector2f add (Vector2f v1, Vector2f v2)
{
    return Vector2f_ctor(v1.x + v2.x, v1.y + v2.y);
}
```

Чтобы **уменьшить** количество аргументов в функции **add** мы можем сделать функцию методом структуры **Vector2f**

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add (Vector2f v)
    {
        return Vector2f_ctor(x + v.x, y + v.y);
    }
};
```

Метод структуры - **функция**, принадлежащая **экземплярам** этой структуры

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add (Vector2f v)
    {
        return Vector2f_ctor(x + v.x, y + v.y);
    }
};
```



Обращение к полям экземпляра структуры

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add (Vector2f v)
    {
        return Vector2f_ctor(x + v.x, y + v.y);
    }
};

int main()
{
    Vector2f v1 = Vector2f_ctor(1, 1);
    Vector2f v2 = Vector2f_ctor(2, 2);
    v1 = v1.add(v2);
}
```



Вызов метода структуры

Задание

Сделать остальные функции методами структуры

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
mul(normalize(add(add(v1, v2), v3)),  
      length(add(v2, v3)))
```

Выражение **до** использования методов
структуры **Vector2f**

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(v1.add(v2).add(v3)).mul(  
    length(v2.add(v3)))
```

Выражение **после** использования методов
структуры **Vector2f**

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(v1.add(v2).add(v3)).mul(  
    length(v2.add(v3)))
```

Выражение упростилось незначительно

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add(Vector2f v);
    Vector2f sub(Vector2f v);
    Vector2f mul(float n);
    Vector2f div (float n);

    Vector2f scalar (Vector2f v);
};
```

Текущее описание структуры вектора

Реализация библиотеки

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f operator+ (Vector2f v);
    Vector2f operator- (Vector2f v);
    Vector2f operator* (float n);
    Vector2f operator/ (float n);

    Vector2f operator* (Vector2f v);
};
```

Перегрузка математических операторов

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(v1.add(v2).add(v3)).mul(  
    length(v2.add(v3)))
```

Выражение **до** перегрузки операторов

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(v1 + v2 + v3) * length(v2 + v3)
```

Выражение **после** перегрузки операторов

Реализация библиотеки

```
Vector2f v1 = Vector_ctor(5, 0);  
Vector2f v2 = Vector_ctor(3, 4);  
Vector2f v3 = Vector_ctor(1, 2);
```

```
normalize(v1 + v2 + v3) * length(v2 + v3)
```

$$v1 = \begin{pmatrix} 5 \\ 0 \end{pmatrix} \quad v2 = \begin{pmatrix} 3 \\ 4 \end{pmatrix} \quad v3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

normalize(v1 + v2 + v3) * len(v2 + v3)



Перегрузка операторов

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add(Vector2f v);
};

int main()
{
    Vector2f v1, v2;
    Vector2f v3 = v1 + v2;
}
```

Перегрузка операторов - это **переопределение**
действия операторов

Перегрузка операторов

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f add(Vector2f v);
};

int main()
{
    Vector2f v1, v2;
    Vector2f v3 = v1 + v2;
}
```



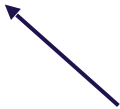
Встретив такой **оператор**
компилятор выдаст ошибку
так как оператор плюс для типа Vector2f
не определен

Перегрузка операторов

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f operator+(Vector2f v);
};

int main()
{
    Vector2f v1, v2;
    Vector2f v3 = v1 + v2;
}
```



Но если **определить** для структуры **оператор**
то компилятор выполнит то, что описано в
operator+

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;
};
```

```
Vector2f Vector2f_ctor (float x, float y)
{
    Vector2f v = {x, y};
    return v;
}
```

Функция **Vector2f_ctor** можно также сделать методом структуры

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f Vector2f_ctor (float x, float y)
    {
        Vector2f v = {x, y};
        return v;
    }
};
```

Функция **Vector2f_ctor** можно также сделать методом структуры

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f Vector2f_ctor (float x, float y)
    {
        Vector2f v = {x, y};
        return v;
    }
};

int main()
{
    Vector2f v;
    v = v.Vector2f_ctor(3, 5);
}
```

Но пользоваться ей станет неудобно

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f(float x, float y)
    {
        this -> x = x;
        this -> y = y;
    }
};

int main()
{
    Vector2f v;
    v = v.Vector2f_ctor(3, 5);
}
```

Чтобы это исправить метод **Vector2f_ctor** можно сделать **конструктором** структуры

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;
    Vector2f(float x, float y)
    {
        this -> x = x;
        this -> y = y;
    }
};

int main()
{
    Vector2f v;
    v = v.Vector2f_ctor(3, 5);
}
```



Имя конструктора должно совпадать с именем структуры

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f(float x, float y)
    {
        this -> x = x;
        this -> y = y;
    }
};

int main()
{
    Vector2f v;
    v = v.Vector2f_ctor(3, 5);
}
```



Конструктор не имеет возвращаемого типа

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f(float x, float y)
    {
        this -> x = x;
        this -> y = y;
    }
};

int main()
{
    Vector2f v1(3, 5);
    Vector2f v2 = Vector2f(3, 5);
}
```



Конструктор легко вызвать

Конструктор

```
struct Vector2f
{
    float x = 0;
    float y = 0;

    Vector2f(float x, float y)
    {
        this -> x = x;
        this -> y = y;
    }
};

int main()
{
    Vector2f v1(3, 5);
    Vector2f v2 = Vector2f(3, 5);
}
```



Указатель на сам экземпляр этой структуры

Структура вектора

```
struct Vector2f  
{
```

```
    float x = 0;  
    float y = 0;
```

```
    Vector2f(float x, float y);
```

```
    Vector2f operator+ (Vector2f v);  
    Vector2f operator- (Vector2f v);  
    Vector2f operator* (Vector2f v);
```

```
    Vector2f operator* (float n);  
    Vector2f operator/ (float n);
```

```
    float length (Vector2f v);
```

```
};
```

Структура **Vector2f**

Структура вектора

```
struct Vector2f
{
    public:

        float x = 0;
        float y = 0;

        Vector2f(float x, float y);

        Vector2f operator+ (Vector2f v);
        Vector2f operator- (Vector2f v);
        Vector2f operator* (Vector2f v);

        Vector2f operator* (float n);
        Vector2f operator/ (float n);

        float length (Vector2f v);
};
```

Структура **Vector2f** с явным указанием, что все поля и методы доступны извне

C++ time

Структура вектора

```
struct Vector2f
{
    public:

        float x = 0;
        float y = 0;

        Vector2f(float x, float y);

        Vector2f operator+ (Vector2f v);
        Vector2f operator- (Vector2f v);
        Vector2f operator* (Vector2f v);

        Vector2f operator* (float n);
        Vector2f operator/ (float n);

        float length (Vector2f v);
};
```

C++ time

Структура вектора

```
class Vector2f
{
    public:

        float x = 0;
        float y = 0;

        Vector2f(float x, float y);

        Vector2f operator+ (Vector2f v);
        Vector2f operator- (Vector2f v);
        Vector2f operator* (Vector2f v);

        Vector2f operator* (float n);
        Vector2f operator/ (float n);

        float length (Vector2f v);
};
```

C++ time

Задание

Рефакторинг математики идеального газа с использованием библиотеки Vector2f

Задание

Рефакторинг структуры Ball